



Engenharia de Computação
Inteligência Artificial
06 - Lógica de Primeira Ordem
Parte II - Inferência em LPO

Organização da aula

- Retomada: linguagem, quantificadores, fatos, regras e consultas em LPO.
- Instanciação universal e existencial.
- Substituição, modus ponens generalizado e unificação.
- Cláusulas definidas e exploração passo a passo por encadeamento direto (*forward chaining*).
- Exploração passo a passo por encadeamento reverso (*backward chaining*) e retrocesso (*backtracking*).
- Resolução em LPO como fechamento e ponte para Prolog.

Fio condutor

Como passar de regras gerais com variáveis para uma conclusão concreta que responda a uma consulta?

Por que inferência em LPO exige um passo a mais?

- Na lógica proposicional, símbolos como P e Q já são unidades fechadas.
- Em LPO, fórmulas podem conter variáveis e quantificadores.
- Antes de aplicar certas regras de inferência, precisamos descobrir **quais objetos** tornam expressões compatíveis.

Ideia central

Grande parte da inferência em LPO depende de encontrar e aplicar **substituições** adequadas.

Substituição

Uma substituição θ associa variáveis a termos.

$$\theta = \{x/\text{John}, y/\text{Mary}\}$$

Para $P(x, y)$:

$$SUBST(\theta, P(x, y)) = P(John, Mary)$$

- o termo substituto pode ser uma constante, variável ou termo funcional;
- a substituição deve ser aplicada de forma consistente em toda a expressão.

Instanciação existencial - IE

De uma afirmação existencial podemos introduzir uma nova constante que representa algum objeto que satisfaz a propriedade.

$$\frac{\exists v \alpha}{SUBST(\{v/k\}, \alpha)}$$

com k uma constante nova para a KB. **Exemplo:**

$$\exists x (Crown(x) \wedge OnHead(x, John))$$

code original

$$Crown(C_1) \wedge OnHead(C_1, John).$$

Uma possibilidade: proposicionalizar a KB

- 1 usar instanciação para gerar versões sem variáveis das sentenças;
- 2 tratar cada sentença básica como um símbolo proposicional;
- 3 aplicar algoritmos de inferência proposicional.

Exemplo:

$$\forall x (King(x) \wedge Greedy(x) \Rightarrow Evil(x))$$

gera, para *John*:

$$King(John) \wedge Greedy(John) \Rightarrow Evil(John).$$

Por que não proposicionalizar tudo?

- podem ser produzidas muitas instâncias irrelevantes para a consulta;
- símbolos funcionais permitem termos de profundidade crescente:

Father(John), Father(Father(John)), ...

- precisamos de mecanismos que trabalhem diretamente com variáveis e estruturas de termos.

Limite teórico

A consequência lógica em LPO é semi-decidível: quando uma consequência existe, há procedimentos capazes de encontrá-la; em geral, não há garantia de término para todas as respostas negativas.

Modus ponens generalizado - MPG

Ideia

Se uma substituição torna os antecedentes de uma regra compatíveis com fatos da KB, podemos inferir o consequente com a mesma substituição.

$$p'_1, \dots, p'_n, (p_1 \wedge \dots \wedge p_n \Rightarrow q)$$

com $SUBST(\theta, p'_i) = SUBST(\theta, p_i)$, então:

$$SUBST(\theta, q)$$

Exemplo de MPG

$$\forall x (King(x) \wedge Greedy(x) \Rightarrow Evil(x))$$

$$King(John) \quad Greedy(John)$$

Uma substituição possível é:

$$\theta = \{x/John\}$$

Logo:

$$Evil(John)$$

Pergunta

O ponto difícil é aplicar o modus ponens ou descobrir a substituição θ ?

Unificação

Definição

Unificar duas expressões significa encontrar uma substituição que as torne idênticas.

$$UNIFY(p, q) = \theta$$

tal que

$$SUBST(\theta, p) = SUBST(\theta, q).$$

Exemplo:

$$UNIFY(Knows(John, x), Knows(John, Jane)) = \{x/Jane\}.$$

Unificador mais geral - UMG

Considere:

$Knows(John, x)$ $Knows(y, z)$

Uma unificação é:

$\{y/John, x/z\}$

Outra, mais específica, seria:

$\{y/John, x/John, z/John\}$

Observação

O **unificador mais geral** impõe o mínimo de restrições necessário; outras unificações podem ser obtidas a partir dele.

Padronização de variáveis

Duas regras podem reutilizar nomes de variáveis sem que essas variáveis sejam o mesmo objeto lógico.

Knows(x, Elizabeth)

Knows(John, x)

Antes de unificar, podemos renomear a primeira variável:

Knows(z, Elizabeth)

e então:

$$\theta = \{z/John, x/Elizabeth\}.$$

Observação

Essa operação é chamada de padronização à parte (*standardize apart*).

Quando a unificação falha?

- símbolos de predicado diferentes: $P(x)$ e $Q(x)$;
- constantes incompatíveis: $Knows(John, x)$ e $Knows(Mary, Bob)$ no primeiro argumento;
- estruturas incompatíveis de termos;
- teste de ocorrência (*occurs check*): impedir $x = f(x)$.

Pergunta

Por que permitir $x = f(x)$ criaria uma substituição problemática?

Atividade em classe - unificação

Em duplas - 6 minutos

Encontre um UMG para cada par ou indique por que a unificação falha.

- 1 $P(x, a)$ e $P(b, y)$
- 2 $Knows(John, x)$ e $Knows(y, Mary)$
- 3 $Parent(x, x)$ e $Parent(John, Mary)$
- 4 $P(x)$ e $P(f(x))$

Depois

Qual desses exemplos exige o teste de ocorrência?

Cláusulas definidas

Uma cláusula definida possui exatamente um literal positivo e pode ser escrita como regra:

$$P_1 \wedge P_2 \wedge \cdots \wedge P_n \Rightarrow Q$$

ou como fato, quando não há antecedentes:

$$Q$$

Observação

Essa forma é particularmente conveniente para encadeamento e será importante na transição para Prolog.

Exemplo condutor: o caso West

Considere a seguinte base de conhecimento:

Fatos

$American(West)$ $Missile(M1)$ $Owns(Nono, M1)$
 $Sells(West, M1, Nono)$ $Enemy(Nono, America)$

Regras

$R_1 : Missile(x) \Rightarrow Weapon(x)$
 $R_2 : Enemy(x, America) \Rightarrow Hostile(x)$
 $R_3 : American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$

Pergunta

A consulta é $Criminal(West)$. Como chegar a essa conclusão explorando a KB?

Duas formas de explorar a base de conhecimento

- **Exploração direta:** partir dos fatos conhecidos e produzir novas conclusões.
- **Exploração indireta/reversa:** partir da conclusão desejada e descobrir quais fatos precisam ser provados.

	Direta	Reversa
Ponto de partida	fatos	consulta/meta
Pergunta principal	“o que posso concluir?”	“o que preciso provar?”
Nome clássico	<i>forward chaining</i>	<i>backward chaining</i>

Conexão com busca

Nos dois casos existe exploração de alternativas. O que muda é o ponto a partir do qual a busca é conduzida.

Exploração direta - encadeamento direto

- começa pelos fatos conhecidos;
- procura regras cujos antecedentes possam ser satisfeitos;
- unifica variáveis, gera novas conclusões e as adiciona à KB;
- repete até obter a consulta ou atingir um ponto fixo.

Observação

É uma estratégia orientada pelos dados (*data-driven*).

Encadeamento direto - passo 0: registrar o que já sabemos

Consulta:

Criminal(West) ?

Base inicial de fatos:

$$F_0 = \{ \textit{American}(\textit{West}), \textit{Missile}(M1), \textit{Owns}(\textit{Nono}, M1), \\ \textit{Sells}(\textit{West}, M1, \textit{Nono}), \textit{Enemy}(\textit{Nono}, \textit{America}) \}$$

Antes de aplicar qualquer regra

Verifique quais regras possuem antecedentes que já podem ser satisfeitos usando somente F_0 .

Pergunta

É possível aplicar R_3 imediatamente? Qual antecedente ainda não está disponível?

Encadeamento direto - passo 2: produzir *Hostile(Nono)*

Temos:

Enemy (Nono, America)

e a regra:

$$R_2 : \text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$$

1. Unificação:

Enemy(x, America) com *Enemy(Nono, America)*

2. Substituição:

$$\theta_2 = \{x / \text{Nono}\}$$

3. Nova conclusão:

Hostile(Nono)

Pergunta

Depois desse passo, todos os antecedentes de R_3 podem ser satisfeitos?

Encadeamento direto - passo 3: provar *Criminal(West)*

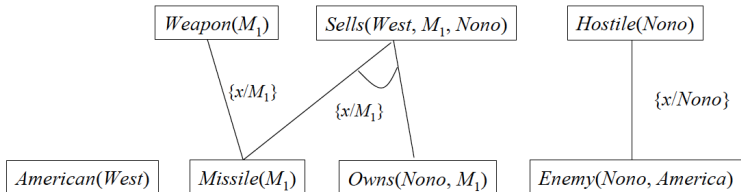
Encadeamento direto - registro completo da exploração

Passo	Regra	Substituição	Novo fato
0	—	—	fatos iniciais
1	R_1	$\{x/M1\}$	$Weapon(M1)$
2	R_2	$\{x/Nono\}$	$Hostile(Nono)$
3	R_3	$\{x/West, y/M1, z/Nono\}$	$Criminal(West)$

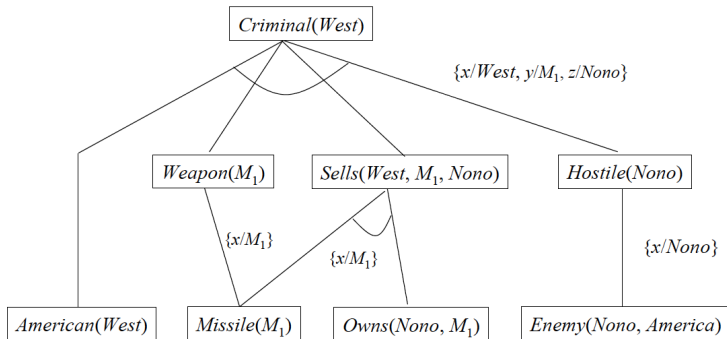
Como resolver no papel

Em cada passo, registre: **regra escolhida** → **unificação** → **substituição** → **novo fato**.

Encadeamento direto no caso West



Árvore de derivação - encadeamento direto



Pergunta

Quais fatos precisaram ser derivados antes de *Criminal(West)* aparecer?

Encadeamento direto: visão do algoritmo

- 1 listar os fatos inicialmente conhecidos;
- 2 seleccionar uma regra cujos antecedentes possam ser satisfeitos;
- 3 encontrar uma substituição que unifique os antecedentes com fatos da KB;
- 4 aplicar a substituição ao conseqüente da regra;
- 5 inserir a nova conclusão na KB, caso ela ainda não esteja presente;
- 6 repetir até obter a consulta ou até nenhuma nova conclusão ser produzida.

Observação

Para cláusulas definidas, o encadeamento direto é correto e completo para as conseqüências expressáveis nesse fragmento.

Quando o encadeamento direto é adequado?

- há muitos fatos chegando ao sistema;
- queremos manter consequências atualizadas;
- há várias consultas possíveis;
- o domínio possui regras acionadas naturalmente pelos dados.

Discussão

Qual é o custo de derivar fatos que nunca serão relevantes para a consulta atual?

Exploração indireta/reversa - encadeamento reverso

- começa pela consulta que desejamos provar;
- procura fatos ou regras cujo conseqüente possa unificar com a meta;
- transforma os antecedentes dessas regras em novas submetas;
- continua até encontrar fatos que sustentem todas as submetas.

Observação

É uma estratégia orientada pela meta (*goal-driven*).

Encadeamento reverso - passo 0: começar pela meta

A consulta é:

Criminal(West)

Pergunta 1: existe esse fato diretamente na KB?

Não.

Pergunta 2: existe uma regra cujo consequente possa unificar com a meta?

$R_3 : \dots \Rightarrow \textit{Criminal}(x)$

A unificação:

$\textit{Criminal}(x)$ com $\textit{Criminal}(\textit{West})$

produz:

$\theta_1 = \{x/\textit{West}\}$

Encadeamento reverso - passo 1: gerar submetas

Aplicando $\theta_1 = \{x/West\}$ em R_3 , a meta *Criminal(West)* é substituída por:

American(West)

Weapon(y)

Sells(West, y, z)

Hostile(z)

Mudança de perspectiva

Em vez de perguntar “o que mais posso concluir?”, passamos a perguntar “o que preciso demonstrar para que a meta seja verdadeira?”.

Encadeamento reverso - passo 2: resolver as primeiras submetas

Submeta 1:

American(West)

Ela é um fato da KB. Portanto, está provada.

Submeta 2:

Weapon(y)

Não há um fato *Weapon(·)* inicialmente, mas R_1 possui consequente compatível:

Missile(u) $\Rightarrow$ Weapon(u)

Logo, surge uma nova submeta:

Missile(y)

O fato *Missile(M1)* unifica com ela:

$\theta_2 = \{y/M1\}$

Encadeamento reverso - passo 3: propagar as substituições

Com $y = M1$, a próxima submeta deixa de ser genérica:

$$\text{Sells}(\text{West}, y, z) \Rightarrow \text{Sells}(\text{West}, M1, z)$$

Na KB temos:

$$\text{Sells}(\text{West}, M1, \text{Nono})$$

Portanto:

$$\theta_3 = \{z/\text{Nono}\}$$

A última submeta torna-se:

$$\text{Hostile}(\text{Nono})$$

Pergunta

Existe esse fato diretamente ou precisamos aplicar outra regra?

Encadeamento reverso - passo 4: fechar a prova

Para provar:

Hostile(Nono)

usamos:

$R_2 : \text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$

Unificando o conseqüente com a submeta:

$\{x / \text{Nono}\}$

surge a nova submeta:

Enemy(Nono, America)

que é um fato da KB.

Todas as submetas foram provadas. Portanto:

Criminal(West)

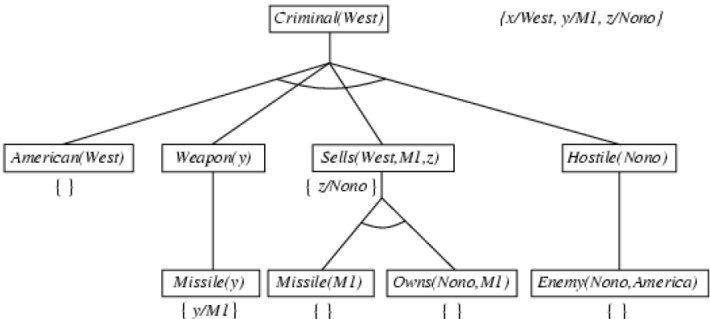
Encadeamento reverso - registro completo da exploração

Passo	Meta/submeta	Como resolver	Resultado
0	<i>Criminal(West)</i>	R_3	4 submetas
1	<i>American(West)</i>	fato	sucesso
2	<i>Weapon(y)</i>	$R_1 + \textit{Missile}(M1)$	$y = M1$
3	<i>Sells(West, M1, z)</i>	fato	$z = \textit{Nono}$
4	<i>Hostile(Nono)</i>	$R_2 + \textit{Enemy}(Nono, America)$	sucesso

Como resolver no papel

Registre: **meta atual** → **regra/fato escolhido** → **substituição** → **novas submetas**.

Árvore de prova - encadeamento reverso



Encadeamento direto versus reverso

	Direto	Reverso
Orientação	dados	meta
Início	fatos conhecidos	consulta
Expansão	regras aplicáveis	regras que podem provar a meta
Registro	fatos derivados	metas/submetas
Risco	derivar fatos irrelevantes	explorar alternativas que falham
Uso típico	regras reativas, múltiplas consultas	consultas específicas, Prolog

Resolução em LPO: unificar antes de cancelar

Consider:

$$\neg \text{Animal}(F(x)) \vee \text{Loves}(G(x), x)$$

$$\neg \text{Loves}(u, v) \vee \text{Kills}(u, v)$$

Para resolver, precisamos unificar:

$$\text{Loves}(G(x), x) \quad \text{e} \quad \text{Loves}(u, v)$$

com

$$\theta = \{u/G(x), v/x\}.$$

O resolvente é:

$$\neg \text{Animal}(F(x)) \vee \text{Kills}(G(x), x).$$

Prova por refutação - ideia mínima

Suponha que desejamos provar *Mortal(Socrates)* a partir de:

$$\forall x (Human(x) \Rightarrow Mortal(x))$$

Human(Socrates)

Adicionamos:

$$\neg \text{Mortal}(\text{Socrates})$$

A FNC contém:

$$\neg \text{Human}(x) \vee \text{Mortal}(x), \quad \text{Human}(\text{Socrates}), \quad \neg \text{Mortal}(\text{Socrates}).$$

Unificando $x/Socrates$ e resolvendo, obtemos a cláusula vazia $\square$.

Atividade de fechamento - qual estratégia usar?

Em duplas ou trios - 7 minutos

Para cada cenário, escolha encadeamento direto ou reverso e justifique.

- 1 Um sistema recebe continuamente fatos de sensores e deve disparar alertas.
- 2 Um usuário pergunta se um indivíduo específico satisfaz uma propriedade.
- 3 Há milhares de fatos, mas somente uma consulta pontual.
- 4 Queremos materializar previamente consequências para diversas consultas futuras.

Conexão

Qual dessas estratégias se aproxima mais do funcionamento declarativo que veremos em Prolog?

Dúvidas



Dúvida é o começo da sabedoria.